



aggregations.sql*

```
64      4
65      ) AS return_rate
66 FROM order_items oi
67 JOIN products p ON oi.product_id = p.product_id
68 GROUP BY p.category
69 ORDER BY return_rate DESC;
70
71 SELECT
72     customer_id,
73     customer_name,
74     customer_type
75 FROM customers
76 LIMIT 10;
```

	customer_id	customer_name	customer_type
1	C0001	Aryan Maharaj	VIP
2	C0002	Pahal Balay	REGULAR
3	C0003	Rushil Saini	REGULAR
4	C0004	Harini Mall	VIP
5	C0005	Gunbir Parmer	PREMIUM
6	C0006	Farhan Memon	REGULAR
7	C0007	Farid Kaul	REGULAR

Execution finished without errors.

Result: 10 rows returned in 39ms

At line 71:

```
SELECT
    customer_id,
```



cohort_analysis.sql

```
1  -- 12. Year-over-Year Comparison
2  WITH MonthlyRevenue AS (
3      SELECT
4          CAST(strftime('%Y', o.order_date) AS INT) AS r_year,
5          CAST(strftime('%m', o.order_date) AS INT) AS r_month,
6          SUM(oi.quantity * oi.unit_price * (1 - oi.discount_percent / 100.0)) AS revenue
7      FROM order_items oi
8      JOIN orders o ON oi.order_id = o.order_id
9      GROUP BY r_year, r_month
10 )
11 SELECT
12     curr.r_year AS year,
13     curr.r_month AS month,
```

	product_a	product_b	times_bought_together
1	Aut	Est	8
2	Aut	Animi	7
3	Est	Animi	7
4	Aut	Doloribus	6
5	Est	Dicta	6
6	Ipsam	Quas	6
7	Quam	Eum	6

Execution finished without errors.

Result: 10 rows returned in 86ms

At line 118:

-- 16. Self-Join with Window Function (Frequently Bought Together Pairs)

WITH ProductPairs AS (



schema.sql*

```
35 );
36
37 -- 4. Order Items Table
38 CREATE TABLE order_items (
39     item_id TEXT PRIMARY KEY,
40     order_id TEXT NOT NULL,
41     product_id TEXT NOT NULL,
42     quantity INTEGER NOT NULL,
43     unit_price REAL NOT NULL CHECK(unit_price >= 0),
44     discount_percent REAL NOT NULL CHECK(discount_percent BETWEEN 0 AND 100),
45     FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE,
46     FOREIGN KEY (product_id) REFERENCES products(product_id)
47 );
```

	name
1	customers
2	products
3	orders
4	order_items

Execution finished without errors.
Result: 4 rows returned in 29ms
At line 48:
SELECT name FROM sqlite_master WHERE type='table';



window_functions.sql

```

1  -- 7. Running Totals with Window Functions
2  -- Calculate running total of revenue per region, ordered by date
3  WITH DailyRegionRevenue AS (
4      SELECT
5          o.region_code,
6          strftime('%Y-%m-%d', o.order_date) AS order_day,
7          SUM(oi.quantity * oi.unit_price * (1 - oi.discount_percent / 100.0)) AS daily_revenue
8      FROM order_items oi
9      JOIN orders o ON oi.order_id = o.order_id
10     GROUP BY o.region_code, order_day
11 )
12 SELECT
13     region_code,
14     order_day

```

	customer_id	total_value	quartile	quartile_label
1	C0059	56693.96	1	Platinum
2	C0197	54066.27	1	Platinum
3	C0415	45400.54	1	Platinum
4	C0180	44952.18	1	Platinum
5	C0215	44487.92	1	Platinum
6	C0116	42383.43	1	Platinum

Execution finished without errors.

Result: 432 rows returned in 37ms

At line 101:

-- 11. NTILE for Segmentation

-- Divide customers into 4 quartiles based on lifetime value

WITH CustomerLTV AS (

File

Edit

Selection

View

Go

...

←

→

WEEK 8

EXPLORER

...

WEEK 8

data

cleaned

products_clean.csv

raw

customers.csv

order_items.csv

orders.csv

products.csv

ecommerce_analytics.db

output

data_quality_report.txt

screenshots

scripts

__pycache__

clean_data.py

generate_data.py

load_to_db.py

report_cli.py

test_constraints.py

sql

aggregations.sql

cohort_analysis.sql

schema.sql

window_functions.sql

tests

venv

README.md

requirements.txt

OUTLINE

TIMELINE

generate_data.py

clean_data.py

load_to_db.py

report_cli.py

test_constraints.py

README.md

scripts > clean_data.py > ...

```
10
11 def apply_clean(file_name, id_col, clean_func=None):
12     """Utility to load, deduplicate, and perform specific cleaning logic."""
13     print(f"\nProcessing {file_name}...")
14     df = pd.read_csv(os.path.join(RAW_DIR, file_name))
15
16     # 1. Structural Cleaning: Force ID to string and drop duplicates
17     df[id_col] = df[id_col].astype(str).str.strip()
18     initial_len = len(df)
19     df = df.drop_duplicates(subset=[id_col], keep='first')
20     print(f" Dropped {initial_len - len(df)} duplicates.")
21     print(f" Final row count: {len(df)}")
22
23     # 2. Content Cleaning: Apply custom logic (dates, names, NULLs)
24     if clean_func:
25         df = clean_func(df)
26
27     df.to_csv(os.path.join(CLEANED_DIR, file_name.replace(".csv", "_clean.csv")), index=False)
28     return df
29
30 def clean_customers():
31     def logic(df):
32         # Requirements: Fill missing emails
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
Processing orders.csv...
Dropped 50 duplicates.
Final row count: 1000

Processing order_items.csv...
Dropped 100 duplicates.
Final row count: 4000
Dropped 0 orphaned records.

All datasets cleaned successfully and saved to /data/cleaned/
(venv) PS D:\CELEBAL\WEEK 8>
```

powersh...

Python

Python

Python

Python

In 71, Col 11

Spaces: 4

File

Edit

Selection

View

Go

...

←

→

WEEK 8

EXPLORER

...

WEEK 8

data

cleaned

products_clean.csv

raw

customers.csv

order_items.csv

orders.csv

products.csv

ecommerce_analytics.db

output

data_quality_report.txt

screenshots

scripts

__pycache__

clean_data.py

generate_data.py

load_to_db.py

report_cli.py

test_constraints.py

sql

aggregations.sql

cohort_analysis.sql

schema.sql

window_functions.sql

tests

venv

README.md

requirements.txt

OUTLINE

TIMELINE

generate_data.py

clean_data.py

load_to_db.py

report_cli.py

test_constraints.py

README.md

scripts

generate_data.py

...

1 import os

2 import random

3 import datetime

4 import pandas as pd

5 from faker import Faker

6

7 # Configuration

8 fake = Faker("en_IN")

9 random.seed(42)

10 Faker.seed(42)

11

12 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

13 RAW_DATA_PATH = os.path.join(BASE_DIR, "data", "raw")

14 os.makedirs(RAW_DATA_PATH, exist_ok=True)

15

16 # Dataset Sizes

17 NUM_CUSTOMERS = 500

18 NUM_PRODUCTS = 500

19 NUM_ORDERS = 1000

20 NUM_ORDER_ITEMS = 4000

21

22 def _save_to_csv(df, file_name):

...

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Processing order_items.csv...

Dropped 100 duplicates.

Final row count: 4000

Dropped 0 orphaned records.

All datasets cleaned successfully and saved to /data/cleaned/

(venv) PS D:\CELEBAL\WEEK 8> & "d:\CELEBAL\WEEK 8\venv\Scripts\python.exe" "d:/CELEBAL/WEEK 8/scripts/generate_data.py"

Generated customers.csv: 550 records.

Generated products.csv: 550 records.

Generated orders.csv: 1050 records.

Generated order_items.csv: 4100 records.

(venv) PS D:\CELEBAL\WEEK 8>

+

...

|

[]

x

powersh...

Python

Python

Python

Python

Ln 15, Col 1

Spaces: 4

UTF-8

CRLF

{ }

Python

venv (3.11.7)

